

Atelier pratique : Maîtriser Hadoop HDFS avec Docker

Module Big Data

Objectif pédagogique

Cet atelier a pour objectif de permettre aux étudiants de maîtriser les bases de Hadoop HDFS à travers un cluster lancé avec Docker Compose. L'atelier se concentre uniquement sur HDFS : stockage distribué, fichiers, dossiers, blocs, réplication et tolérance aux pannes.

Table des matières

1	Tableau récapitulatif des commandes HDFS	3
2	Résumé des colonnes utilisées dans l'atelier	4
3	Objectifs de l'atelier	4
4	Pré-requis	4
5	Présentation de l'architecture du cluster	5
5.1	Rôle du NameNode	5
5.2	Rôle des DataNodes	5
5.3	Rôle de la réplication	6
6	Fichier docker-compose.yml	6
7	Fichier de configuration Hadoop	7
7.1	Explication des paramètres	7
8	Lancement du cluster	8
9	Accès à l'interface web du NameNode	8

10 Connexion au conteneur NameNode	9
11 Activité 1 : vérifier l'état du cluster HDFS	9
12 Activité 2 : créer une arborescence HDFS	9
13 Activité 3 : créer des fichiers CSV locaux	10
14 Activité 4 : envoyer des fichiers vers HDFS	11
15 Activité 5 : copier, déplacer et supprimer des fichiers HDFS	11
16 Activité 6 : télécharger des fichiers HDFS vers le système local	12
17 Activité 7 : analyser la taille des fichiers	13
18 Activité 8 : comprendre les blocs HDFS	13
18.1 Interprétation du résultat	13
19 Activité 9 : modifier le facteur de réplication	14
20 Activité 10 : simuler la panne d'un DataNode	14
21 Exercice de synthèse	15
21.1 Objectif	15
21.2 Travail demandé	15
21.3 Fichier à créer	16
21.4 Livrables	16
22 Problèmes fréquents et solutions	16
22.1 Le NameNode ne démarre pas	16
22.2 Les DataNodes n'apparaissent pas	16
22.3 L'interface web ne fonctionne pas	17
22.4 Erreur lors de l'ajout d'un fichier déjà existant	17

1. Tableau récapitulatif des commandes HDFS

Avant de commencer les manipulations, le tableau suivant résume les commandes HDFS les plus utilisées dans cet atelier.

Commande HDFS	Rôle
<code>hdfs dfs -ls /</code>	Afficher le contenu de la racine HDFS.
<code>hdfs dfs -ls /chemin</code>	Afficher le contenu d'un dossier HDFS.
<code>hdfs dfs -ls -R /chemin</code>	Afficher récursivement toute l'arborescence d'un dossier.
<code>hdfs dfs -mkdir /dossier</code>	Créer un dossier dans HDFS.
<code>hdfs dfs -mkdir -p /a/b/c</code>	Créer une arborescence complète dans HDFS.
<code>hdfs dfs -put fichier /chemin</code>	Envoyer un fichier local vers HDFS.
<code>hdfs dfs -put -f fichier /chemin</code>	Envoyer un fichier local vers HDFS en remplaçant l'ancien fichier s'il existe.
<code>hdfs dfs -get /chemin/fichier</code>	Télécharger un fichier depuis HDFS vers le système local.
<code>hdfs dfs -cat /chemin/fichier</code>	Afficher tout le contenu d'un fichier HDFS.
<code>hdfs dfs -head /chemin/fichier</code>	Afficher le début d'un fichier HDFS.
<code>hdfs dfs -cp source destination</code>	Copier un fichier ou un dossier dans HDFS.
<code>hdfs dfs -mv source destination</code>	Déplacer ou renommer un fichier dans HDFS.
<code>hdfs dfs -rm fichier</code>	Supprimer un fichier dans HDFS.
<code>hdfs dfs -rm -r dossier</code>	Supprimer un dossier HDFS avec son contenu.
<code>hdfs dfs -du -h /chemin</code>	Afficher la taille des fichiers dans un format lisible.
<code>hdfs dfs -count /chemin</code>	Compter les dossiers, fichiers et la taille totale.
<code>hdfs dfs -setrep -w 3 fichier</code>	Modifier le facteur de réplication d'un fichier.
<code>hdfs dfsadmin -report</code>	Afficher l'état général du cluster HDFS.
<code>hdfs fsck /chemin -files -blocks -locations</code>	Analyser les fichiers, les blocs et leurs emplacements.

Remarque

Les commandes commençant par `hdfs dfs` servent à manipuler les fichiers et les dossiers dans HDFS. Les commandes `hdfs dfsadmin` et `hdfs fsck` servent plutôt à l'administration et au diagnostic du cluster.

2. Résumé des colonnes utilisées dans l'atelier

Dans cet atelier, nous allons manipuler des fichiers CSV représentant des ventes mensuelles. Ces fichiers seront stockés dans HDFS afin de simuler une petite zone de stockage distribuée.

Nom de la colonne	Type de donnée	Description
id_vente	Entier	Identifiant unique de chaque vente.
date_vente	Date	Date de réalisation de la vente.
client	Texte	Nom du client.
ville	Texte	Ville où la vente a été effectuée.
categorie	Texte	Catégorie du produit vendu.
produit	Texte	Nom du produit vendu.
quantite	Entier	Quantité vendue.
prix_unitaire	Décimal	Prix unitaire du produit.
montant	Décimal	Montant total de la vente.

Remarque

Les fichiers CSV utilisés dans cet atelier servent uniquement à apprendre à manipuler les fichiers dans HDFS. L'objectif n'est pas encore de faire du traitement de données avec MapReduce, Spark ou Hive.

3. Objectifs de l'atelier

À la fin de cet atelier, l'étudiant doit être capable de :

1. Lancer un cluster Hadoop HDFS avec Docker Compose.
2. Comprendre le rôle du NameNode et des DataNodes.
3. Manipuler HDFS avec les commandes `hdfs dfs`.
4. Créer une arborescence HDFS.
5. Ajouter, lire, copier, déplacer et supprimer des fichiers dans HDFS.
6. Comprendre la notion de bloc HDFS.
7. Modifier et vérifier le facteur de réplication.
8. Simuler la panne d'un DataNode.
9. Analyser l'état du cluster avec `dfsadmin` et `fsck`.
10. Comprendre l'intérêt de HDFS dans une architecture Big Data.

4. Pré-requis

Avant de commencer l'atelier, il faut avoir installé :

- Docker ;

- Docker Compose ;
- un terminal ;
- un éditeur de texte.

Vérifier l'installation de Docker :

```
docker --version
docker compose version
```

Créer un dossier de travail :

```
mkdir atelier-hadoop-hdfs
cd atelier-hadoop-hdfs
```

L'arborescence du dossier doit être la suivante :

```
atelier-hadoop-hdfs/
|
|-- docker-compose.yml
|-- config
|-- volumes/
```

5. Présentation de l'architecture du cluster

Le cluster Hadoop utilisé dans cet atelier contient :

- un **NameNode** ;
- cinq **DataNodes**.

5.1. Rôle du NameNode

Le NameNode est le composant principal de HDFS. Il ne stocke pas directement les fichiers des utilisateurs. Il stocke les métadonnées :

- les noms des fichiers ;
- les chemins des fichiers ;
- les blocs associés aux fichiers ;
- les DataNodes qui contiennent les blocs ;
- les informations de réplication.

5.2. Rôle des DataNodes

Les DataNodes stockent réellement les blocs de données. Lorsqu'un fichier est envoyé vers HDFS, il est découpé en blocs, puis ces blocs sont distribués et répliqués sur plusieurs DataNodes.

5.3. Rôle de la réplication

La réplication permet de conserver plusieurs copies d'un même bloc sur différents DataNodes. Si un DataNode tombe en panne, les données restent accessibles à partir des autres DataNodes.

Remarque

Dans cet atelier, nous utilisons Docker pour simuler un cluster Hadoop sur une seule machine. Dans un environnement réel, les DataNodes sont généralement répartis sur plusieurs serveurs physiques ou machines virtuelles.

6. Fichier docker-compose.yml

Le fichier suivant permet de lancer un cluster Hadoop HDFS avec un NameNode et cinq DataNodes.

```
services:
  namenode:
    image: apache/hadoop:3.3.6
    hostname: namenode
    command: ["hdfs", "namenode"]
    ports:
      - "9870:9870"
      - "8020:8020"
    env_file:
      - ./config
    environment:
      ENSURE_NAMENODE_DIR: "/tmp/hadoop-root/dfs/name"
    volumes:
      - ./volumes/namenode:/data

  datanode1:
    image: apache/hadoop:3.3.6
    hostname: datanode1
    command: ["hdfs", "datanode"]
    env_file:
      - ./config

  datanode2:
    image: apache/hadoop:3.3.6
    hostname: datanode2
    command: ["hdfs", "datanode"]
    env_file:
      - ./config

  datanode3:
    image: apache/hadoop:3.3.6
    hostname: datanode3
    command: ["hdfs", "datanode"]
    env_file:
```

```
- ./config

datanode4:
  image: apache/hadoop:3.3.6
  hostname: datanode4
  command: ["hdfs", "datanode"]
  env_file:
    - ./config

datanode5:
  image: apache/hadoop:3.3.6
  hostname: datanode5
  command: ["hdfs", "datanode"]
  env_file:
    - ./config
```

Remarque

Si votre fichier `docker-compose.yml` contient aussi `resourcemanager` et `nodemanager`, vous pouvez les garder. Ils ne seront pas utilisés dans cet atelier, car nous ne faisons pas de MapReduce.

7. Fichier de configuration Hadoop

Créer un fichier nommé `config`, sans extension :

```
touch config
```

Ajouter le contenu suivant :

```
HADOOP_HOME=/opt/hadoop

CORE-SITE.XML_fs.defaultFS=hdfs://namenode:8020

HDFS-SITE.XML_dfs.namenode.rpc-address=namenode:8020
HDFS-SITE.XML_dfs.replication=3
HDFS-SITE.XML_dfs.blocksize=134217728
HDFS-SITE.XML_dfs.webhdfs.enabled=true
```

7.1. Explication des paramètres

- `fs.defaultFS=hdfs://namenode:8020` : indique que HDFS est le système de fichiers par défaut.
- `dfs.namenode.rpc-address=namenode:8020` : indique l'adresse du NameNode.
- `dfs.replication=3` : indique que chaque bloc doit être répliqué trois fois.
- `dfs.blocksize=134217728` : indique que la taille d'un bloc HDFS est de 128 Mo.
- `dfs.webhdfs.enabled=true` : active l'accès WebHDFS.

8. Lancement du cluster

Lancer le cluster avec Docker Compose :

```
docker compose up -d
```

Vérifier les conteneurs :

```
docker compose ps
```

Afficher les logs du NameNode :

```
docker compose logs namenode
```

Afficher les logs d'un DataNode :

```
docker compose logs datanode1
```

Les conteneurs attendus sont :

```
namenode  
datanode1  
datanode2  
datanode3  
datanode4  
datanode5
```

9. Accès à l'interface web du NameNode

Ouvrir le navigateur et accéder à l'adresse suivante :

```
http://localhost:9870
```

Cette interface permet de consulter :

- l'état général du cluster ;
- la capacité totale ;
- l'espace utilisé ;
- l'espace disponible ;
- les DataNodes actifs ;
- les fichiers stockés dans HDFS ;
- les informations sur les blocs.

Remarque

L'interface web du NameNode est très utile pour visualiser rapidement la santé du cluster HDFS et vérifier si les DataNodes sont bien connectés.

10. Connexion au conteneur NameNode

Toutes les commandes HDFS seront exécutées depuis le conteneur **namenode**.

```
docker compose exec namenode bash
```

Vérifier la version de Hadoop :

```
hadoop version
```

Tester HDFS :

```
hdfs dfs -ls /
```

Remarque

Si HDFS est vide, la commande peut ne rien afficher. Ce comportement est normal au début de l'atelier.

11. Activité 1 : vérifier l'état du cluster HDFS

Afficher le rapport HDFS :

```
hdfs dfsadmin -report
```

Cette commande affiche :

- la capacité totale du cluster ;
- la capacité utilisée ;
- la capacité restante ;
- le nombre de DataNodes vivants ;
- le nombre de DataNodes morts ;
- les informations détaillées sur chaque DataNode.

Cette étape permet de vérifier que le cluster HDFS est bien opérationnel avant de commencer les manipulations sur les fichiers.

12. Activité 2 : créer une arborescence HDFS

Créer un dossier principal pour l'atelier :

```
hdfs dfs -mkdir /atelier
```

Créer des sous-dossiers :

```
hdfs dfs -mkdir /atelier/input  
hdfs dfs -mkdir /atelier/output  
hdfs dfs -mkdir /atelier/archive  
hdfs dfs -mkdir /atelier/logs
```

Afficher le contenu de HDFS :

```
hdfs dfs -ls /  
hdfs dfs -ls /atelier
```

Créer une arborescence de type Data Lake simple :

```
hdfs dfs -mkdir -p /datalake/ventes/raw  
hdfs dfs -mkdir -p /datalake/ventes/processed  
hdfs dfs -mkdir -p /datalake/ventes/archive
```

Afficher l'arborescence complète :

```
hdfs dfs -ls -R /datalake
```

Remarque

L'option `-p` permet de créer automatiquement les dossiers parents s'ils n'existent pas. Par exemple, la commande `hdfs dfs -mkdir -p /datalake/ventes/raw` crée `/datalake`, `/datalake/ventes` et `/datalake/ventes/raw` si nécessaire.

13. Activité 3 : créer des fichiers CSV locaux

Créer un dossier local temporaire dans le conteneur :

```
mkdir -p /tmp/datasets
```

Créer le fichier `ventes_janvier.csv` :

```
cat > /tmp/datasets/ventes_janvier.csv << 'EOF'  
id_vente,date_vente,client,ville,categorie,produit,quantite,prix_unitaire,montant  
1,2026-01-03,Ahmed,Casablanca,Informatique,Ordinateur,1,8500,8500  
2,2026-01-05,Fatima,Rabat,Bureautique,Imprimante,1,2300,2300  
3,2026-01-09,Youssef,Fes,Accessoire,Clavier,2,175,350  
4,2026-01-15,Sara,Marrakech,Accessoire,Souris,3,60,180  
5,2026-01-21,Omar,Casablanca,Informatique,Ecran,1,2200,2200  
EOF
```

Créer le fichier `ventes_fevrier.csv` :

```
cat > /tmp/datasets/ventes_fevrier.csv << 'EOF'  
id_vente,date_vente,client,ville,categorie,produit,quantite,prix_unitaire,montant  
6,2026-02-02,Nadia,Tanger,Informatique,Ordinateur,1,7900,7900  
7,2026-02-06,Hamza,Rabat,Bureautique,Scanner,1,1600,1600  
8,2026-02-11,Imane,Agadir,Mobile,Tablette,1,3200,3200  
9,2026-02-17,Karim,Casablanca,Mobile,Smartphone,1,4500,4500  
10,2026-02-23,Hajar,Fes,Informatique,DisqueSSD,2,700,1400  
EOF
```

Créer le fichier `ventes_mars.csv` :

```
cat > /tmp/datasets/ventes_mars.csv << 'EOF'
id_vente,date_vente,client,ville,categorie,produit,quantite,prix_unitaire,montant
11,2026-03-04,Ali,Casablanca,Informatique,PCPortable,1,9200,9200
12,2026-03-08,Mouna,Rabat,Accessoire,Casque,2,350,700
13,2026-03-13,Rachid,Tanger,Mobile,Smartphone,1,5100,5100
14,2026-03-19,Salma,Marrakech,Bureautique,Photocopieur,1,12000,12000
15,2026-03-25,Mehdi,Agadir,Accessoire,Souris,5,80,400
EOF
```

Vérifier les fichiers locaux :

```
ls -lh /tmp/datasets
cat /tmp/datasets/ventes_janvier.csv
```

Remarque

Ces fichiers sont créés localement dans le conteneur **namenode**. Ils seront ensuite envoyés vers HDFS avec la commande **hdfs dfs -put**.

14. Activité 4 : envoyer des fichiers vers HDFS

Envoyer les fichiers CSV vers la zone **raw** :

```
hdfs dfs -put /tmp/datasets/ventes_janvier.csv /datalake/ventes/raw/
hdfs dfs -put /tmp/datasets/ventes_fevrier.csv /datalake/ventes/raw/
hdfs dfs -put /tmp/datasets/ventes_mars.csv /datalake/ventes/raw/
```

Vérifier le contenu de la zone **raw** :

```
hdfs dfs -ls /datalake/ventes/raw
```

Lire un fichier directement depuis HDFS :

```
hdfs dfs -cat /datalake/ventes/raw/ventes_janvier.csv
```

Afficher uniquement le début du fichier :

```
hdfs dfs -head /datalake/ventes/raw/ventes_janvier.csv
```

Remarque

La commande **-put** copie un fichier local vers HDFS. La commande **-cat** affiche le contenu complet du fichier. La commande **-head** affiche uniquement le début du fichier.

15. Activité 5 : copier, déplacer et supprimer des fichiers HDFS

Copier les fichiers de la zone **raw** vers la zone **archive** :

```
hdfs dfs -cp /datalake/ventes/raw/*.csv /datalake/ventes/archive/
```

Vérifier :

```
hdfs dfs -ls /datalake/ventes/archive
```

Créer un fichier de test :

```
echo "fichier temporaire" > /tmp/test.txt  
hdfs dfs -put /tmp/test.txt /atelier/input/
```

Déplacer ce fichier vers le dossier archive :

```
hdfs dfs -mv /atelier/input/test.txt /atelier/archive/
```

Vérifier le déplacement :

```
hdfs dfs -ls /atelier/input  
hdfs dfs -ls /atelier/archive
```

Supprimer le fichier :

```
hdfs dfs -rm /atelier/archive/test.txt
```

Vérifier :

```
hdfs dfs -ls /atelier/archive
```

Remarque

La commande `-cp` copie un fichier ou un dossier dans HDFS. La commande `-mv` déplace un fichier ou le renomme. La commande `-rm` supprime un fichier HDFS.

16. Activité 6 : télécharger des fichiers HDFS vers le système local

Créer un dossier local de sortie :

```
mkdir -p /tmp/export
```

Télécharger un fichier depuis HDFS :

```
hdfs dfs -get /datalake/ventes/raw/ventes_janvier.csv /tmp/export/
```

Vérifier localement :

```
ls -lh /tmp/export  
cat /tmp/export/ventes_janvier.csv
```

Remarque

La commande `-get` permet de récupérer un fichier depuis HDFS vers le système local du conteneur. Le fichier reste disponible dans HDFS après le téléchargement.

17. Activité 7 : analyser la taille des fichiers

Afficher la taille des fichiers dans HDFS :

```
hdfs dfs -du -h /datalake/ventes/raw
```

Afficher le nombre de fichiers et dossiers :

```
hdfs dfs -count /datalake/ventes/raw
```

Afficher l'arborescence complète :

```
hdfs dfs -ls -R /datalake/ventes
```

Remarque

La commande `-du -h` permet d'afficher la taille des fichiers dans un format lisible. La commande `-count` affiche le nombre de dossiers, le nombre de fichiers et la taille totale.

18. Activité 8 : comprendre les blocs HDFS

Analyser les blocs du fichier `ventes_janvier.csv` :

```
hdfs fsck /datalake/ventes/raw/ventes_janvier.csv -files -blocks -locations
```

Analyser toute la zone `raw` :

```
hdfs fsck /datalake/ventes/raw -files -blocks -locations
```

Remarque

Les fichiers utilisés dans cet atelier sont petits. Chaque fichier sera probablement stocké dans un seul bloc HDFS. Pour de très gros fichiers, HDFS découpe automatiquement les données en plusieurs blocs. Chaque bloc est ensuite stocké et répliqué sur plusieurs DataNodes selon le facteur de réplication configuré.

18.1. Interprétation du résultat

La commande `hdfs fsck` permet d'observer :

- le chemin du fichier ;
- la taille du fichier ;

- le nombre de blocs ;
- le facteur de réplication ;
- les DataNodes qui contiennent les blocs ;
- l'état général du fichier.

19. Activité 9 : modifier le facteur de réplication

Afficher les informations de réplication :

```
hdfs fsck /datalake/ventes/raw/ventes_janvier.csv -files -blocks -locations
```

Changer le facteur de réplication à 2 :

```
hdfs dfs -setrep -w 2 /datalake/ventes/raw/ventes_janvier.csv
```

Vérifier :

```
hdfs fsck /datalake/ventes/raw/ventes_janvier.csv -files -blocks -locations
```

Changer le facteur de réplication à 3 pour les trois fichiers :

```
hdfs dfs -setrep -w 3 /datalake/ventes/raw/ventes_janvier.csv  
hdfs dfs -setrep -w 3 /datalake/ventes/raw/ventes_fevrier.csv  
hdfs dfs -setrep -w 3 /datalake/ventes/raw/ventes_mars.csv
```

Vérifier toute la zone raw :

```
hdfs fsck /datalake/ventes/raw -files -blocks -locations
```

Remarque

Un facteur de réplication égal à 3 signifie que chaque bloc est stocké en trois exemplaires sur le cluster. Cela permet d'améliorer la disponibilité des données en cas de panne d'un DataNode.

20. Activité 10 : simuler la panne d'un DataNode

Sortir du conteneur NameNode :

```
exit
```

Arrêter un DataNode :

```
docker compose stop datanode5
```

Retourner dans le conteneur NameNode :

```
docker compose exec namenode bash
```

Vérifier l'état du cluster :

```
hdfs dfsadmin -report
```

Lire un fichier malgré l'arrêt du DataNode :

```
hdfs dfs -cat /datalake/ventes/raw/ventes_janvier.csv
```

Analyser les blocs :

```
hdfs fsck /datalake/ventes/raw -files -blocks -locations
```

Redémarrer le DataNode :

```
exit  
docker compose start datanode5
```

Vérifier à nouveau :

```
docker compose exec namenode bash  
hdfs dfsadmin -report
```

Remarque

Lorsque le fichier est répliqué sur plusieurs DataNodes, l'arrêt d'un DataNode ne provoque pas immédiatement une perte de données. HDFS continue à servir les fichiers à partir des autres copies disponibles sur les autres DataNodes.

21. Exercice de synthèse

21.1. Objectif

Cet exercice permet de vérifier la maîtrise des commandes essentielles de HDFS.

21.2. Travail demandé

Réaliser les tâches suivantes :

1. Créer le dossier `/exercice`.
2. Créer les sous-dossiers `/exercice/raw`, `/exercice/archive` et `/exercice/export`.
3. Créer localement un fichier CSV nommé `clients.csv`.
4. Envoyer le fichier `clients.csv` dans `/exercice/raw`.
5. Lire le fichier depuis HDFS.
6. Copier le fichier vers `/exercice/archive`.
7. Télécharger le fichier depuis HDFS vers le système local.
8. Afficher la taille du fichier avec `du -h`.
9. Vérifier les blocs du fichier avec `fsck`.
10. Modifier le facteur de réplication à 3.

21.3. Fichier à créer

```
cat > /tmp/clients.csv << 'EOF'
id_client,nom,ville,pays
1,Ahmed,Casablanca,Maroc
2,Fatima,Rabat,Maroc
3,Youssef,Fes,Maroc
4,Sara,Marrakech,Maroc
EOF
```

21.4. Livrables

Chaque étudiant doit rendre :

- les commandes utilisées ;
- une capture de l'interface web du NameNode ;
- le résultat de la commande `hdfs dfs -ls -R /exercice` ;
- le résultat de la commande `hdfs fsck /exercice/raw/clients.csv -files -blocks -locations` ;
- une courte explication sur le rôle de la réplication.

22. Problèmes fréquents et solutions

22.1. Le NameNode ne démarre pas

Afficher les logs :

```
docker compose logs namenode
```

Redémarrer le cluster :

```
docker compose down
docker compose up -d
```

Si le problème vient d'un ancien état du cluster, supprimer les volumes locaux :

```
docker compose down
rm -rf volumes
docker compose up -d
```

Remarque

La commande `rm -rf volumes` supprime les données persistées localement. Elle doit être utilisée avec prudence.

22.2. Les DataNodes n'apparaissent pas

Vérifier les logs :


```
docker compose logs datanode1
docker compose logs datanode2
docker compose logs datanode3
docker compose logs datanode4
docker compose logs datanode5
```

Vérifier le fichier config :

```
CORE-SITE.XML_fs.defaultFS=hdfs://namenode:8020
HDFS-SITE.XML_dfs.namenode.rpc-address=namenode:8020
```

22.3. L'interface web ne fonctionne pas

Vérifier que le port suivant existe dans le fichier `docker-compose.yml` :

```
ports:
  - "9870:9870"
```

Puis ouvrir :

```
http://localhost:9870
```

22.4. Erreur lors de l'ajout d'un fichier déjà existant

Si vous envoyez deux fois le même fichier avec `-put`, HDFS peut afficher une erreur car le fichier existe déjà.

Solution 1 : supprimer l'ancien fichier puis envoyer à nouveau le fichier.

```
hdfs dfs -rm /datalake/ventes/raw/ventes_janvier.csv
hdfs dfs -put /tmp/datasets/ventes_janvier.csv /datalake/ventes/raw/
```

Solution 2 : utiliser l'option `-f` pour remplacer le fichier.

```
hdfs dfs -put -f /tmp/datasets/ventes_janvier.csv /datalake/ventes/raw/
```